DESIGN AND ANALYSIS OF ALGORITHMS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE CODE & NAME:

CSA06 – DESIGN AND ANALYSIS OF ALGORITHMS

ASSESSMENT: CO4-AT1-ASSIGNMENT 4A –Algorithmic Design Through Pseudocode

Course: Design and Analysis of Algorithms

Course Outcome: CO4 – Solve optimization problems using Dynamic

Programming and Greedy strategies Assignment nature: Individual academic report

Submitted by: S.Harshavardhan

Registration Number:192524106

B.Tech – Artificial Intelligence and Data Science

SIMATS Engineering

INTRODUCTION

This report presents complete answers for all six tasks in Assignment 4A. The answers follow the problem statements and data supplied in the assignment. The main focus is on interpreting a requirement, identifying inputs and outputs, defining a decision or state, writing precise language-independent pseudocode, tracing the algorithm, and testing the design.

An algorithm is a finite sequence of clear steps that solves a problem. Pseudocode expresses those steps in a structured form without depending on a particular programming language. It should be detailed enough that another person can implement it without guessing what words such as “best”, “suitable”, or “process” mean.

OBJECTIVES OF THIS REPORT

The report explains a fractional knapsack greedy method, job sequencing with deadlines, the 0/1 knapsack dynamic programming method, the correction of a Word Break defect, three shortest-path approaches, and weighted interval scheduling for an energy-aware data centre.

For each task, the solution includes the required reasoning and not only the final result. Where the assignment asks for a manual trace or challenge test, the report shows the important decisions and state changes. All headings are in black. No page number, header, or footer is added.

TASK 1 – ALGORITHM DESIGN REVIEW

1.1 Understanding the problem

The logistics company has a vehicle with a fixed carrying capacity. Every package has a weight and a value. Packages may be divided, so a fraction of a package can be loaded. The aim is to obtain the maximum possible total value without making the total loaded weight greater than the vehicle capacity. This is the fractional knapsack problem.

The important feature is that an item is divisible. Therefore, if the remaining vehicle capacity is smaller than the weight of the next package, the algorithm can load only the fraction that fits. This is different from the 0/1 knapsack problem, where an item must be selected completely or rejected.

Inputs and outputs

Input: a list of packages, where each package has an identifier, weight, and value; and the vehicle capacity C. Each package weight must be positive, and its value must be non-negative for the usual formulation.

Output: the fraction of each package loaded, the total weight loaded, and the maximum total value. The total weight must not exceed C.

Greedy decision

The greedy rule is to consider packages in descending order of value per unit weight. The ratio value/weight measures how much value is obtained from one unit of vehicle capacity. Taking the highest ratio first gives the greatest immediate value for each unit of available capacity.

TASK 1 – DIAGNOSIS OF POOR PSEUDOCODE

1.2 Task 1.1 – Algorithmic deficiencies

Deficiency 1 – Input is not defined. “Read packages” does not state that each package has an identifier, weight, and value, and it does not read the vehicle capacity. An implementer cannot know which data must be supplied.

Deficiency 2 – “Arrange packages” has no ordering rule. The algorithm does not say whether packages are arranged by weight, value, or value per unit weight. Different orders can produce different total values.

Deficiency 3 – “Select the best package” is unclear. The word “best” does not define a measurable criterion. A precise algorithm must state the comparison, which is the value-to-weight ratio.

Deficiency 4 – The feasibility test is incomplete. “Vehicle has space” does not compare the next package's weight with the remaining capacity. The algorithm must track remaining capacity numerically.

Deficiency 5 – Divisibility is ignored. A package may be divided, but “Add suitable package” may be understood as adding only a complete package. The algorithm must calculate the fraction that fits when the remaining capacity is insufficient for a whole package.

Deficiency 6 – Remaining capacity is not updated. After loading a whole package or a fraction, the algorithm must subtract the loaded weight from the remaining capacity. Otherwise, it may exceed the vehicle limit.

Deficiency 7 – Termination is not precise. The loop should stop when capacity becomes zero or all packages have been considered. The phrase “vehicle has space” does not explain what happens when all packages have been checked.

Deficiency 8 – Result construction is missing. The algorithm does not record the fraction of each package loaded or explain how total value is accumulated. “Calculate value” is not enough to reconstruct the selected load.

TASK 1 – REPAIRED PSEUDOCODE

1.3 Task 1.2 – Complete greedy algorithm

The corrected design first calculates the value-to-weight ratio for every package. It sorts packages from the highest ratio to the lowest ratio. It then loads complete packages while possible and takes a fraction of the next package when only part of its weight can fit.

Let C be the vehicle capacity. Let remaining be the capacity not yet used. For package i, let weight[i] and value[i] be its weight and value. The loaded fraction is between zero and one.

```
ALGORITHM FractionalKnapsack(Packages,
C)
INPUT: Packages with ID, weight and value;
capacity C
OUTPUT: Loaded fraction of each package and maximum total value

BEGIN FOR each package i in
Packages DO ratio[i] <-
value[i] / weight[i] fraction[i]
<- 0 END FOR

SORT Packages by ratio in
descending order remaining <- C

totalValue <- 0

FOR each package i in sorted Packages DO
  IF remaining = 0 THEN
    BREAK
  END IF

  IF weight[i] <= remaining THEN fraction[i]
<- 1 remaining <- remaining - weight[i]
totalValue <- totalValue + value[i] ELSE
fraction[i] <- remaining / weight[i] totalValue
<- totalValue + fraction[i] * value[i] remaining
<- 0 END IF
END FOR
```

```
    RETURN fraction, C - remaining, totalValue
  END
```

The algorithm records every selected fraction. If a complete package fits, its full value is added. Otherwise, the fraction that fits is loaded and the loop ends because no capacity remains.

TASK 1 – DESIGN DEFENCE AND CORRECTNESS

1.4 Task 1.3 – Why this is pseudocode

This solution is pseudocode because it uses structured steps such as FOR, IF, SORT, and RETURN to describe the algorithm without using the syntax of a particular programming language. It is more precise than informal English because each decision, update, and stopping condition is stated. It does not depend on a compiler, data type declaration, or language-specific library. A programmer can translate the steps into any suitable programming language.

Greedy-choice explanation

Suppose two packages have ratios r1 and r2, where r1 is greater than r2. For the same amount of available capacity, a fraction of the first package gives more value than the same weight of the second package. Therefore, choosing the highest ratio first is the best immediate use of capacity. Since packages are divisible, the algorithm can fill any remaining capacity with a fraction of the next package.

The divisibility condition is essential to this reasoning. For indivisible packages, choosing the largest ratio first is not always optimal because a package may not fit and combinations of complete packages may be better. That separate problem is addressed in Task 3.

Complexity

Calculating one ratio for each of n packages takes O(n) time. Sorting the packages takes O(n log n) time. The loading loop examines each package at most once and takes O(n) time. Therefore, the total time complexity is O(n log n), dominated by sorting. The algorithm stores a ratio and a loaded fraction for each package, so its auxiliary storage is O(n).

Conclusion: the repaired algorithm is complete because it defines the data, ordering rule, feasibility check, fractional selection, capacity update, termination, and returned result.

TASK 2 – JOB SEQUENCING WITH DEADLINES

2.1 Task 2.1 – Model the requirement

The cloud-service company receives independent jobs. Each job needs exactly one processing slot. A job earns its profit only if it is completed on or before its deadline. Only one job can occupy a slot. The objective is to choose and arrange jobs so that the total profit is as large as possible.

Input: a set of jobs. Each job has a unique ID, a positive integer deadline, and a profit. Output: the scheduled jobs, the slot assigned to each scheduled job, and the total profit.

Objective: maximise the sum of profits of all jobs that are scheduled within their deadlines.

Constraints: every job uses one slot; a slot can contain at most one job; a job assigned to slot t must satisfy t less than or equal to its deadline; and no job can be scheduled more than once.

Decision: for each job, decide whether to include it and, if included, which available slot to assign. To test feasibility, the algorithm needs the job's deadline and the current availability of slots from 1 through the maximum deadline.

Greedy hypothesis

Greedy rule: process jobs in descending order of profit and assign each job to the latest currently empty slot that is not later than its deadline. This preserves earlier slots for jobs with tighter deadlines and accepts the most profitable jobs first.

TASK 2 – GREEDY PSEUDOCODE

2.2 Task 2.2 – Justification

A high-profit job should be considered before a lower-profit job because the goal is to maximise total profit. Placing an accepted job in the latest free slot before its deadline leaves earlier slots available for jobs with smaller deadlines. The rule combines profit priority with a placement decision that protects scheduling flexibility.

2.3 Task 2.3 – Complete pseudocode

```
ALGORITHM JobSequencing(Jobs)
INPUT: Jobs with ID, deadline and profit
OUTPUT: Slot schedule and maximum total
profit

BEGIN
 maxDeadline <- maximum deadline
among Jobs      FOR slot <- 1 TO
maxDeadline DO
   Schedule[slot] <- EMPTY   END
FOR   totalProfit <- 0   SORT Jobs
by profit in descending order

 FOR each job j in sorted Jobs DO      slot <-
MIN(j.deadline, maxDeadline)      WHILE slot
>= 1 AND Schedule[slot] is not EMPTY DO
slot <- slot - 1     END WHILE

   IF    slot    >=    1    THEN
Schedule[slot] <- j.ID
    totalProfit  <-  totalProfit  +  j.profit
END IF
 END FOR

 RETURN Schedule, totalProfit
END
```

The loop searches backward from the job's deadline. If it finds an empty slot, it places the job there and adds the profit. If no slot from 1 to the deadline is empty, the job is rejected. The schedule is therefore feasible by construction.

TASK 2 – DESK-CHECK

2.4 Task 2.4 – Given instance

The input jobs are A(deadline 2, profit 80), B(1, 35), C(2, 60), D(3, 40), and E(1, 25). The maximum deadline is 3, so the schedule contains slots 1, 2, and 3.

After sorting by descending profit, the order is A, C, D, B, E. The algorithm considers jobs in that order.

Decision 1: Job A has deadline 2. Slot 2 is empty, so A is placed in slot 2. Total profit becomes 80.

Decision 2: Job C has deadline 2. Slot 2 is occupied, so the algorithm checks slot 1. Slot 1 is empty, so C is placed in slot 1. Total profit becomes 140.

Decision 3: Job D has deadline 3. Slot 3 is empty, so D is placed in slot 3. Total profit becomes 180.

Decision 4: Job B has deadline 1. Slot 1 is occupied by C. There is no earlier slot, so B is rejected. Total profit remains 180.

Decision 5: Job E has deadline 1. Slot 1 is occupied, so E is rejected. Total profit remains 180.

Final schedule

Slot 1 = C; Slot 2 = A; Slot 3 = D. The selected jobs are A, C, and D. The maximum profit produced by this trace is 80 + 60 + 40 = 180.

The schedule is valid because each job occupies one slot and each assigned slot is not later than its deadline.

TASK 2 – CHALLENGE AND ANALYSIS

2.5 Task 2.5 – Modified input

Change only the profit of job C from 60 to 90. All deadlines and all other profits remain unchanged. Before tracing, predict that the schedule remains feasible but the order of the first two jobs changes: C is considered before A because its new profit is higher.

The new descending-profit order is C(90), A(80), D(40), B(35), E(25). Job C is placed in slot 2, the latest slot not later than deadline 2. Job A then finds slot 2 occupied and is placed in slot 1. Job D is placed in slot 3. Jobs B and E cannot be placed because slot 1 is occupied and their deadlines are 1.

The final schedule is slot 1 = A, slot 2 = C, slot 3 = D. Total profit = 80 + 90 + 40 = 210. The prediction is correct: the processing order changes, but the same three job IDs are scheduled in the available slots.

Why the modified decision matters

The change affects the priority decision between A and C. Both jobs have deadline 2 and compete for the same two slots. Increasing C's profit makes it the first job considered. Since the algorithm places the first job in the latest available slot, C is assigned slot 2 and A is assigned slot 1.

The test demonstrates that the ordering criterion is active and that the algorithm updates the total profit only when a job is successfully scheduled. It also confirms that a job with deadline 1 cannot be placed in slot 2 or 3, even when those slots are empty.

Conclusion: the greedy algorithm uses an explicit profit order, a backward slot search, a feasibility test, and a profit update. The desk-check shows a feasible schedule and a total profit of 180 for the original data.

TASK 3 – DYNAMIC PROGRAMMING: 0/1 KNAPSACK

3.1 Task 3.1 – Define the state

The items in this task are indivisible. Each item must either be selected completely or not selected. Let there be n items and let the vehicle capacity be W. Each item i has weight weight[i] and value value[i].

Define DP[i][w] as the maximum total value that can be obtained using only the first i items, with total weight not exceeding capacity w. The first index tells how many items are available for consideration. The second index tells the capacity limit of the subproblem.

This state is useful because the best answer for a larger set of items can be built from answers to smaller sets. At item i, there are only two choices: exclude item i or include it if its weight fits.

Boundary conditions

DP[0][w] = 0 for every capacity w, because no items are available. DP[i][0] = 0 for every i, because a vehicle with zero capacity cannot carry any positive-weight item. These values form the starting row and column of the DP table.

The final optimum value is DP[n][W], because it considers all n items and the full vehicle capacity.

TASK 3 – RECURRENCE RELATION

3.2 Task 3.2 – Include and exclude decisions

If the current item i is heavier than the available capacity w, it cannot be included. In that case, the best value is the same as the best value using the previous i−1 items and capacity w.

If weight[i] is less than or equal to w, two possibilities must be compared. Excluding item i gives DP[i−1][w]. Including item i gives value[i] plus DP[i−1][w−weight[i]]. The second term represents the best value that can be obtained from earlier items using the capacity left after including item i.

The recurrence is:

DP[0][w] = 0 for 0 <= w <= W

DP[i][0] = 0 for 0 <= i <= n

If weight[i] > w:

DP[i][w] = DP[i-1][w]

Otherwise:

```
DP[i][w] = MAX( DP[i-1][w], value[i] + DP[i-1][w-weight[i]] )
```

The recurrence uses the previous row i−1 in both choices. This is important because an item may be selected at most once. If the include decision referred to row i again, the same item could be reused, which would describe a different problem.

Meaning of the terms

DP[i−1][w] is the value when item i is rejected. value[i] + DP[i−1][w−weight[i]] is the value when item i is selected. MAX chooses the better of these two feasible alternatives. When the item is too heavy, only the exclude choice is permitted.

TASK 3 – COMPLETE PSEUDOCODE

3.3 Task 3.3 – DP table and reconstruction

```
ALGORITHM ZeroOneKnapsack(Items, W)
INPUT: n items with weight[i], value[i]; capacity W
OUTPUT: Maximum value and selected item IDs

BEGIN
  CREATE DP[0..n][0..W]

  FOR w <- 0 TO W DO
    DP[0][w] <- 0
  END FOR
  FOR i <- 0 TO n DO
    DP[i][0] <- 0
  END FOR

  FOR i <- 1 TO n DO
    FOR w <- 1 TO W DO
      IF weight[i] > w THEN
```

```
      DP[i][w]    <-    DP[i-
1][w]                    ELSE
exclude <- DP[i-1][w]
      include <- value[i] + DP[i-1][w-weight[i]]
DP[i][w] <- MAX(exclude, include)
    END IF
  END
FOR   END
FOR

 optimum              <-
DP[n][W]     selected
<- EMPTY LIST    w
<- W

 FOR i <- n DOWNTO 1 DO
  IF  DP[i][w]  !=  DP[i-1][w]
THEN           ADD  item  i  to
selected      w <- w - weight[i]
END IF
 END FOR

 RETURN optimum, selected
END
```

The table is filled from smaller item counts to larger item counts. At every cell, the previous row is already available. The reconstruction starts at DP[n][W]. If the current cell differs from the cell above it, the current item contributed to the optimum and is selected. Its weight is then subtracted from the remaining capacity.

If two alternatives have equal value, the reconstruction rule above treats equality as exclusion. This still returns an optimal solution, although another equally optimal set may exist.

TASK 3 – DESK-CHECK

3.4 Task 3.4 – Given item data

The items are P(weight 2, value 12), Q(weight 3, value 20), R(weight 4, value 25), and S(weight 5, value 30). The vehicle capacity is 7.

Some useful combinations are: P + Q has weight 5 and value 32; P + R has weight 6 and value 37; P + S has weight 7 and value 42; Q + R has weight 7 and value 45. Q + S has weight 8 and is not feasible. R + S has weight 9 and is not feasible.

The best feasible combination is Q + R. Its total weight is 3 + 4 = 7 and its total value is 20 + 25 = 45. This fills the capacity exactly.

Important DP states

After processing P, DP[1][2] = 12 and DP[1][7] = 12. With P alone, the maximum value is 12 because the item can be included once.

After processing Q, DP[2][5] = 32 because P and Q fit together. DP[2][7] = 32 because no additional item is available and their combined weight is only 5.

After processing R, at capacity 7 the include choice is value[R] + DP[2][3] = 25 + 20 = 45. The exclude choice is DP[2][7] = 32. Therefore DP[3][7] = 45.

After processing S, the include choice at capacity 7 is 30 + DP[3][2] = 30 + 12 = 42. The exclude choice is DP[3][7] = 45. Therefore DP[4][7] = 45.

The final answer is DP[4][7] = 45. Reconstruction selects R and Q, giving the item set {Q, R}.

TASK 3 – DEPENDENCY AND COMPLEXITY

3.5 Task 3.5 – Why earlier states are required

A DP state depends on smaller states. To calculate DP[i][w], the algorithm needs DP[i−1][w] for the exclude choice. If item i fits, it also needs DP[i−1][w−weight[i]] for the include choice. Both states belong to the previous item row.

Therefore, the algorithm processes items from 1 to n and capacities from 1 to W. Before row i is calculated, row i−1 has already been completed. This processing order guarantees that every required value exists when it is used.

The dependency is the reason the recurrence can be evaluated without guessing. Each cell is calculated only from already-computed cells. The initial row and column provide the base cases from which all other values are built.

Complexity analysis

The algorithm fills n multiplied by W table entries. Each entry performs a constant amount of work: a feasibility comparison and, when possible, a maximum of two values. The time complexity is O(nW). The DP table requires O(nW) space. The selected-item reconstruction takes O(n) time after the table is complete.

The value W is a numeric capacity, so this is called pseudo-polynomial time rather than polynomial time in the number of bits needed to represent W. For the assignment instance, the capacity is small and the table method is suitable.

Conclusion: the state has a precise meaning, the recurrence considers both choices, the pseudocode computes the table in dependency order, and reconstruction returns the actual items rather than only the numerical optimum.

TASK 4 – PSEUDOCODE FORENSICS

4.1 Task 4.1 – Locate the defect

The purpose of Word Break is to determine whether the complete input string can be divided into valid dictionary words. Let Text have length n. DP[i] should mean that the prefix consisting of the first i characters can be completely segmented into dictionary words.

The base condition DP[0] = TRUE is correct because the empty prefix requires no words and is considered successfully segmented. All other DP entries are initially FALSE.

The defect is in the inner decision. The given algorithm sets DP[i] to TRUE whenever substring(Text, j, i) is in the dictionary. It does not check whether the prefix ending at j can itself be segmented.

A dictionary word at the end of a prefix is not enough. The part before that word must also be valid. The correct condition must require both DP[j] = TRUE and the substring from j+1 through i to be in the dictionary, using the assignment's prefix-position convention.

Why the missing condition matters

Without DP[j], the algorithm may jump over an invalid beginning of the text. It can find a valid dictionary word near the end and incorrectly mark the whole prefix as valid. A correct dynamic programming transition must connect the new word to a previously valid prefix.

TASK 4 – FAILING INPUT AND TRACE

4.2 Task 4.2 – Counterexample

Choose Text = “cats” and Dictionary = {“at”, “s”}. The string “cats” cannot be segmented completely using these words because the first character “c” is not a dictionary word and no valid first word begins at the start of the string.

Initial state: DP[0] = TRUE. DP[1], DP[2], DP[3], and DP[4] are set to FALSE.

At i = 1, the algorithm checks the one-character prefix. The substring “c” is not in the dictionary, so DP[1] remains FALSE.

At i = 2, the algorithm checks substrings ending at position 2. It finds “at” in the dictionary. The faulty algorithm sets DP[2] = TRUE even though DP[0] is the only valid earlier prefix and the substring selected by the indexing convention does not cover a valid prefix from the beginning. The correct algorithm must not accept a dictionary word unless the preceding prefix is reachable.

At i = 4, the algorithm finds “s” in the dictionary and sets DP[4] = TRUE, even though the prefix before “s” is not valid under the correct state relation. Thus the faulty algorithm returns TRUE for a string that cannot be segmented.

The first incorrect state is the first position at which a dictionary suffix is accepted without confirming that its preceding prefix is valid. The counterexample shows why the prefix-state check is essential.

State interpretation

DP[i] is not simply “some substring ending at i is a dictionary word.” It is “the entire prefix up to i can be segmented.” The second meaning is stronger and is the one needed for the final answer DP[n].

TASK 4 – REPAIR AND REGRESSION

4.3 Task 4.3 – Minimal correction

Only the inner IF condition needs to be corrected. The outer loops, initialisation, and final return remain unchanged. The repaired condition checks that the earlier prefix is already segmentable and that the next substring is a dictionary word.

```
ALGORITHM WordBreak(Text, Dictionary)
BEGIN
  DP[0] <- TRUE
  FOR i <- 1 TO n DO
    DP[i] <- FALSE
    FOR j <- 0 TO i - 1 DO
IF DP[j] = TRUE AND
       substring(Text, j, i) is in Dictionary THEN
DP[i] <- TRUE
       BREAK
     END IF
    END FOR
  END FOR
  RETURN DP[n]
END
```

The expression substring(Text, j, i) represents the portion after the prefix of length j and ending at i. In an implementation, the indexing convention must be applied consistently. The logical repair is the additional DP[j] = TRUE requirement.

4.4 Task 4.4 – Regression check

Use Text = “cats” and Dictionary = {“cat”, “s”}. The correct answer is TRUE because the string can be divided into “cat” and “s”.

The corrected algorithm begins with DP[0] = TRUE. It can set DP[3] = TRUE because the prefix of length zero is valid and “cat” is in the dictionary. It then sets DP[4] = TRUE because DP[3] is TRUE and “s” is in the dictionary. The final result is
TRUE.

The correction preserves this valid case and rejects the earlier invalid case. It therefore repairs the missing logical dependency without replacing the complete algorithm.

Conclusion: a DP state must represent the entire prefix, not merely the existence of a dictionary word somewhere inside it. The added prefix check makes the recurrence correct.

TASK 5 – SHORTEST PATH ALGORITHMS

5.1 Task 5.1 – Directed weighted graph

The graph contains vertices A, B, C, D, and E. Its directed edges are A→B with weight 4, A→C with weight 7, B→C with weight 2, B→D with weight 6, C→D with weight 1, D→E with weight 3, and C→E with weight 8.

The first requirement is to find shortest distances from source A. All edge weights are assumed to be non-negative. Under this assumption, Dijkstra's greedy strategy can be used.

Initialisation: distance[A] = 0. The distance of every other vertex is infinity because no route from A has yet been confirmed. Every predecessor is initially NIL. All vertices begin as unvisited.

At each step, select the unvisited vertex with the smallest tentative distance. Mark it as settled, then examine each outgoing edge. If travelling through the selected vertex gives a smaller distance to a neighbour, update that neighbour's distance and predecessor.

The predecessor information is important because it allows the actual shortest path to be reconstructed, not just its length.

TASK 5 – DIJKSTRA PSEUDOCODE

5.1 Complete greedy design

```
ALGORITHM Dijkstra(Graph, A)
INPUT: Directed graph with non-negative edge weights; source A OUTPUT: Shortest distances and predecessor of each vertex

BEGIN FOR each vertex v DO
distance[v] <- INFINITY
predecessor[v] <- NIL
visited[v] <- FALSE
END FOR distance[A] <- 0

 REPEAT |V| times
   u <- unvisited vertex with minimum distance
IF u does not exist OR distance[u] = INFINITY THEN
    BREAK
END IF
visited[u] <- TRUE

   FOR each edge (u, v) with weight w DO IF visited[v] = FALSE AND distance[u] + w < distance[v] THEN
distance[v] <- distance[u] + w
predecessor[v] <- u END IF
   END FOR
 END REPEAT

 RETURN distance, predecessor
END
```

The selection operation extracts the unvisited vertex with the smallest tentative distance. Relaxation checks whether the route through u is shorter than the best route already known for v. The algorithm terminates after all reachable vertices have been settled or no reachable unvisited vertex remains.

Trace from A

Initially, A = 0 and B, C, D, E = infinity. Select A: update B to 4 with predecessor A, and C to 7 with predecessor A. Select B next because 4 is smallest: update C to 6 through B and D to 10 through B. Select C next: update D to 7 through C and E to 14 through C. Select D next: update E to 10 through D. Select E last; no better distance is found.

TASK 5 – SHORTEST-PATH RESULTS

Dijkstra final result

The final distances from A are: A = 0, B = 4, C = 6, D = 7, and E = 10. The predecessor chain for E is EDCBA, so one shortest path is A→B→C→D→E with total weight 4 + 2 + 1 + 3 = 10.

The direct edge A→C has weight 7, but the route A→B→C has weight 6, so the indirect route is shorter. Similarly, the route to D through C has weight 7, which is better than the route through B with weight 10. The route to E through D has weight 10, which is better than the direct route from C to E after reaching C.

Why the greedy choice is safe

With non-negative edge weights, extending a path cannot reduce its cost. When the unvisited vertex with the smallest tentative distance is selected, no later path through another unvisited vertex can make its distance smaller. This is the property that makes the greedy commitment safe.

If negative-weight edges are allowed, a path discovered later can reduce the distance of a vertex that was already selected. Therefore, Dijkstra's settled-vertex decision is no longer guaranteed to be correct.

Transition to Bellman-Ford

Bellman-Ford does not permanently settle a vertex after one minimum-distance selection. Instead, it repeatedly relaxes all edges. This allows improvements to move through the graph over multiple passes, including when some edges have negative weights.

TASK 5 – BELLMAN-FORD

5.2 Task 5.2 – Negative-weight edges

The unsafe assumption in Dijkstra is that all edge weights are non-negative. The greedy step that selects the smallest tentative distance and permanently marks that vertex as settled depends on this assumption. With a negative edge, a route found later may be cheaper than the distance already settled.

Bellman-Ford starts with distance[source] = 0 and all other distances set to infinity. It performs $|V|-1$ passes over all edges. During each pass, every edge (u, v) with weight w is relaxed if u is reachable and distance[u] + w is smaller than distance[v].

```
ALGORITHM BellmanFord(Graph, source)
BEGIN       FOR   each
vertex        v         DO
distance[v]              <-
INFINITY
predecessor[v]  <-  NIL
END FOR
 distance[source] <- 0

 FOR  pass  <-  1  TO  |V|  -  1  DO
changed <- FALSE     FOR each edge
(u,  v,  w)  DO         IF  distance[u]  !=
INFINITY AND         distance[u] + w
< distance[v] THEN          distance[v]
<- distance[u] + w       predecessor[v]
<- u        changed <- TRUE       END
IF
   END  FOR          IF
changed     =     FALSE
THEN
    BREAK
   END IF
 END FOR
```

After the repeated relaxation passes, examine every edge once more. If a reachable edge can still be relaxed, a reachable negative-weight cycle exists. Such a cycle means there is no finite shortest path for vertices whose path can use that cycle.

TASK 5 – BELLMAN-FORD AND FLOYD

Bellman-Ford completion

```
  FOR each edge (u, v, w) DO      IF
distance[u]   !=   INFINITY   AND
distance[u] + w < distance[v] THEN
RETURN
NEGATIVE_WEIGHT_CYCLE
   END IF
  END FOR

  RETURN distance, predecessor
END
```

The check includes distance[u] not equal to infinity so that a negative cycle in a disconnected component is not incorrectly described as reachable from the chosen source. The algorithm reports a negative cycle only when the cycle can be reached from the source.

5.3 Task 5.3 – All-pairs shortest paths

The organisation now requires shortest paths between every pair of vertices. Define D(k)[i][j] as the shortest distance from vertex i to vertex j when only the first k vertices in a chosen vertex ordering are allowed as intermediate vertices. The start and end vertices may be any vertices; the restriction applies to intermediate vertices.

At stage k, the path from i to j has two possibilities. It may not use vertex k as an intermediate vertex, in which case its distance remains D(k−1)[i][j]. Or it may pass through k, in which case its distance is D(k−1)[i][k] + D(k−1)[k][j].

The transition is the minimum of these two alternatives. This is the central recurrence of Floyd's algorithm.

TASK 5 – FLOYD PSEUDOCODE

Floyd state transition

```
  D <- initial distance matrix
```

```
FOR each vertex i DO
  D[i][i] <- 0
END FOR

FOR k <- 1 TO |V| DO
  FOR i <- 1 TO |V| DO
    FOR j <- 1 TO |V| DO
      D[i][j] <- MIN(
        D[i][j],
        D[i][k] + D[k][j]
      )
    END FOR
  END FOR
END FOR
RETURN D
```

The initial matrix contains zero on the diagonal, the given edge weight for a direct edge, and infinity where no direct edge exists. Each outer-loop iteration permits one more vertex to be used as an intermediate vertex. After all vertices have been considered, D contains the shortest distances between all pairs, provided the graph has no negative cycle.

Difference between the three approaches

Dijkstra solves a single-source problem using a greedy minimum-distance selection and requires non-negative edge weights. Bellman-Ford also solves a single-source problem, but uses repeated edge relaxation and supports negative edges; it can detect reachable negative cycles. Floyd solves an all-pairs problem using a matrix of subproblem states and a dynamic programming transition.

The state in Floyd is indexed by both a source and destination vertex, along with the set of permitted intermediate vertices. This is different from Dijkstra and Bellman-Ford, which maintain one distance value from a fixed source to each vertex.

Conclusion: the same graph can require different algorithmic reasoning depending on the edge assumptions and the output requested.

TASK 6 – ENERGY-AWARE JOB SCHEDULING

Stage 1 – Interpret the problem

A cloud data centre has one specialised energy-efficient server. Several jobs are waiting during a maintenance window. Every job has a start time, a finish time, and an energy-saving value. The server can run only one job at a time, so two jobs whose time intervals overlap cannot both be selected.

The important assumption states that if one job finishes at time t and another starts at time t, the two jobs do not overlap. Therefore, a previous job is compatible with a new job when its finish time is less than or equal to the new job's start time.

Input: a list of jobs with ID, start time, finish time, and energy-saving value. Output: a set of selected job IDs and the maximum total energy-saving value.

Objective: maximise the sum of energy-saving values of selected jobs.

Constraints: the server processes one job at a time; selected intervals must be pairwise non-overlapping; each job is selected at most once; and the finish-time boundary rule allows one job to start exactly when another finishes.

Decision: select or reject each job while considering the total value of compatible combinations. A locally high-value job may block two or more jobs whose combined value is larger.

TASK 6 – STRATEGY SELECTION

Stage 2 – Propose the strategy

The suitable strategy is dynamic programming for weighted interval scheduling. The objective is not simply to select the job that finishes earliest, has the highest value, or has the shortest duration. Each job has a different energy-saving value, and the best answer may be a combination of compatible jobs.

An earliest-finish greedy rule is useful for selecting the largest number of non-overlapping jobs when every job has equal value. It does not necessarily maximise total energy savings when values differ. Selecting the highest-value job first can also fail because that job may overlap several jobs whose combined values are greater.

The dynamic programming method compares two choices for each job after sorting by finish time: include the job and add the best compatible earlier result, or exclude it and keep the best result already found.

Stage 3 – Critical information and state

First, sort the jobs in non-decreasing order of finish time. For each job i, calculate p(i), the index of the last job before i whose finish time is less than or equal to the start time of job i. If no earlier job is compatible, p(i) = 0.

Define M[i] as the maximum total energy-saving value obtainable from the first i jobs in sorted order. The base condition is M[0] = 0 because no jobs produce zero total saving.

The include decision uses value[i] + M[p(i)]. The exclude decision uses M[i−1]. The recurrence is M[i] = MAX(value[i] + M[p(i)], M[i−1]). This state retains the best value for each prefix of the sorted jobs.

TASK 6 – COMPLETE PSEUDOCODE

Stage 4 – Weighted interval scheduling

```
ALGORITHM EnergyAwareScheduling(Jobs)
INPUT: Jobs with ID, start, finish, saving value
OUTPUT: Selected jobs and maximum total saving

BEGIN
  SORT Jobs by finish time in ascending order

  FOR i <- 1 TO n DO     p[i] <-
0     FOR j <- i - 1 DOWNTO 1
DO         IF finish[j] <= start[i]
THEN       p[i] <- j       BREAK
      END IF
    END FOR
  END FOR

  M[0] <- 0     FOR i <- 1 TO n DO
includeValue <- saving[i] + M[p[i]]
excludeValue <- M[i-1]         M[i] <-
MAX(includeValue,      excludeValue)
END FOR

  selected <- EMPTY
LIST   i <- n   WHILE
i > 0 DO
    includeValue <- saving[i] +
M[p[i]]      IF includeValue > M[i-
1] THEN          ADD Jobs[i].ID to
selected       i <- p[i]     ELSE       i
<- i - 1     END IF
  END WHILE

  REVERSE selected
  RETURN selected, M[n]
```

END

The compatibility search scans earlier jobs from nearest to farthest. Since the list is sorted by finish time, the first compatible job found is the latest compatible predecessor. The dynamic programming table then stores the best total for each prefix.

During reconstruction, if including job i gives a strictly larger value than excluding it, the job is selected and the algorithm continues at p(i). Otherwise, it skips the job and moves to i−1. In a tie, this version skips the current job; another tie rule could return a different but equally optimal schedule.

TASK 6 – DATA AND COMPATIBILITY

Given eight jobs

The assignment data are: J1 (1, 3, 20), J2 (2, 5, 50), J3 (4, 6, 30), J4 (6, 8, 40), J5 (5, 9, 60), J6 (8, 10, 30), J7 (9, 11, 50), and J8 (11, 13, 40). Each tuple gives start time, finish time, and energy-saving value.

The jobs are already in non-decreasing finish-time order. The predecessor values are: p(1)=0, p(2)=0, p(3)=1, p(4)=3, p(5)=2, p(6)=4, p(7)=5, and p(8)=7.

For J3, J1 is compatible because J1 finishes at 3 and J3 starts at 4. For J4, J3 is compatible because J3 finishes at 6 and J4 starts at 6. For J5, J2 is compatible because J2 finishes at 5 and J5 starts at 5. For J8, J7 is compatible because J7 finishes at 11 and J8 starts at 11.

The equality cases are accepted because the assignment explicitly says that a job finishing at time t and another starting at time t are non-overlapping.

Why predecessor information is enough

Once p(i) is known, every job after p(i) and before i conflicts with job i under the sorted interval order. Therefore, if job i is selected, the best earlier contribution comes from the first p(i) jobs. If job i is rejected, the best answer comes from the first i−1 jobs.

TASK 6 – DESK-CHECK

Stage 5 – DP table calculation

Use M[0] = 0 and the recurrence M[i] = MAX(saving[i] + M[p(i)], M[i−1]). The values are calculated in sorted order.

J1: include = 20 + M[0] = 20; exclude = 0. Therefore M[1] = 20.

J2: include = 50 + M[0] = 50; exclude = 20. Therefore M[2] = 50.

J3: include = 30 + M[1] = 50; exclude = 50. Therefore M[3] = 50. The values tie, so the reconstruction rule skips J3.

J4: include = 40 + M[3] = 90; exclude = 50. Therefore M[4] = 90.

J5: include = 60 + M[2] = 110; exclude = 90. Therefore M[5] = 110.

J6: include = 30 + M[4] = 120; exclude = 110. Therefore M[6] = 120.

J7: include = 50 + M[5] = 160; exclude = 120. Therefore M[7] = 160.

J8: include = 40 + M[7] = 200; exclude = 160. Therefore M[8] = 200.

The final maximum value is M[8] = 200.

TASK 6 – RECONSTRUCTION AND RESULT

Selected jobs

Start reconstruction at i = 8. For J8, include value is 200 and exclude value is 160, so select J8 and move to p(8) = 7.

At i = 7, include value is 160 and exclude value is 120, so select J7 and move to p(7) = 5.

At i = 5, include value is 110 and exclude value is 90, so select J5 and move to p(5) = 2.

At i = 2, include value is 50 and exclude value is 20, so select J2 and move to p(2) = 0. Reconstruction ends.

The selected jobs are J2, J5, J7, and J8. Their intervals are 2–5, 5–9, 9–11, and 11–13. These jobs do not overlap because each job starts exactly when the previous selected job finishes.

Total energy-saving value = 50 + 60 + 50 + 40 = 200. This matches M[8], so the reconstructed schedule agrees with the dynamic programming optimum.

Decision trace summary

The algorithm does not select J6 in the final schedule even though M[6] = 120 is larger than M[5] = 110. When J7 is considered, the include decision uses M[5], which deliberately skips J6 and combines J7 with the better schedule ending at J5. This shows why the predecessor index is important.

The result is not obtained by simply adding all high-value jobs. It is obtained by comparing compatible combinations and preserving the best result for each earlier prefix.

TASK 6 – ATTACK THE DESIGN

Stage 6 – Adversarial test with five jobs

Construct the following new set of jobs to challenge a greedy rule that selects the job with the highest individual energy-saving value first:

A: start 1, finish 10, value 100. B: start 1, finish 5, value 60. C: start 5, finish 10, value 60. D: start 10, finish 12, value 20. E: start 12, finish 14, value 20.

This input is challenging because job A has the highest individual value, but it overlaps both B and C. The two jobs B and C together provide 120, which is greater than A's value of 100. The boundary rule makes B and C compatible because B finishes at 5 and C starts at 5.

Prediction: a highest-value-first greedy strategy will select A first. It will then reject B and C because they overlap A, but it can still select D and E. Its total will be 100 + 20 + 20 = 140. The dynamic programming strategy should find the better combination B, C, D, and E with total 160.

Applying the DP strategy

The jobs are sorted by finish time: B(5,60), C(10,60), A(10,100), D(12,20), E(14,20). For the same finish time, either order of C and A can be used if the compatibility calculation is applied consistently. The predecessor of C is B because B finishes at 5 and C starts at 5. A has no compatible predecessor among B and C because both overlap A.

TASK 6 – ADVERSARIAL TRACE AND CONCLUSION

DP result for the challenge input

The best value from B is 60. The best value from B and C is 60 + 60 = 120 because they are compatible. Job A alone gives 100, which is less than 120, so the DP comparison rejects A in favour of the B-and-C combination. Job D can follow C or A at time 10, and job E can follow D at time 12.

The optimal selected set is B, C, D, and E. Its total is 60 + 60 + 20 + 20 = 160. The locally attractive choice A would produce only 140 when followed by D and E. Therefore, highest-value-first greedy selection fails on this adversarial input, while the dynamic programming comparison finds the larger compatible combination.

The critical decision is the comparison between including A and using the best compatible earlier combination B and C. The dynamic programming state retains the value of the best result for each prefix, so it does not commit permanently to the highest individual job.

Overall conclusion

This assignment demonstrates two important algorithm design strategies. Greedy algorithms make a locally preferred choice according to a clearly defined rule. They are suitable when the problem structure supports that choice, as in fractional knapsack and the stated job sequencing method. Dynamic programming stores solutions to smaller subproblems and combines them to obtain a global optimum, as in 0/1 knapsack, Word Break, Floyd's algorithm, and weighted interval scheduling.

The report also shows why precise state definitions and feasibility conditions are necessary. A missing condition can create an incorrect result, as shown in the Word Break defect. Manual traces and challenge inputs help test whether the algorithm follows the intended logic.

For the energy-aware scheduling problem, the selected jobs are J2, J5, J7, and J8, giving a maximum energy-saving value of 200. The adversarial test confirms why a combination-based dynamic programming strategy is more appropriate than choosing only the job with the highest individual value.

Final submission note

All six tasks have been answered with problem interpretation, decision or state definitions, pseudocode, traces, and explanations as required by the assignment. The pseudocode is language-independent and avoids relying on a compiler or a particular programming language.